

1 chapter4 动态规划

当子问题不独立时，如果能够保存已解决的子问题的答案，而在需要时再找出已求得的答案，就可以避免大量重复计算，从而得到多项式时间算法。而动态规划算法只解决每个子问题一次，然后将其答案保存起来，从而避免了每次遇到子问题时重新计算答案的工作。即动态规划通过空间开销来节省由于重复计算带来的时间开销。

最优性原理：如果问题的最优解所包含的子问题的解也是子问题的最优解，就称该问题具有最优子结构，即满足最优性原理（优化原则）。

并不是所有优化问题都适用优化原则，有时对某个子问题的解不一定最优，但延伸成整个问题的解时反而最优。

在证明一个问题满足最优子结构性性质时，可以考虑反证法，我们后面在 0-1 背包问题中具体展开。

1.1 最长公共子序列

要证明最长公共子序列问题满足最优子结构性性质，设 $X = \langle x_1, \dots, x_m \rangle$ 和 $Y = \langle y_1, \dots, y_n \rangle$ 的 LCS 为 $Z = \langle z_1, \dots, z_k \rangle$ 。当 $x_m = y_n$ 时必有 $z_k = x_m$ ，否则可将 x_m 追加到 Z 得到更长子序列导致矛盾，此时 Z_{k-1} 必须是 X_{m-1} 和 Y_{n-1} 的 LCS，若存在更长的 W 则 $W \oplus x_m$ 将形成更长公共子序列；当 $x_m \neq y_n$ 时，若 $z_k \neq x_m$ 则 Z 是 X_{m-1} 和 Y 的 LCS，若 $z_k \neq y_n$ 则 Z 是 X 和 Y_{n-1} 的 LCS，否则可通过选择更长子序列构造矛盾。由此 LCS 的最优解总由子问题最优解构成，满足最优子结构性性质。

$s_{[-1]}$ 表示序列 s 的最后一个元素， $s_{[: -1]}$ 表示 s 去掉最后一个元素后的子序列

$$\text{LCS}(s_1, s_2) = \begin{cases} 0, & \text{if } s_1 \text{ or } s_2 \text{ is empty} \\ \text{LCS}(s_{1[: -1]}, s_{2[: -1]}) + 1, & \text{if } s_{1[-1]} = s_{2[-1]} \\ \max(\text{LCS}(s_{1[: -1]}, s_2), \text{LCS}(s_1, s_{2[: -1]})), & \text{otherwise} \end{cases}$$

```
1 while (cin >> s1 >> s2){
2     memset(dp, 0, sizeof(dp));
3     int n1 = strlen(s1), n2 = strlen(s2);
4     for (int i = 1; i <= n1; ++i)
5         for (int j = 1; j <= n2; ++j)
6             if (s1[i - 1] == s2[j - 1])
7                 dp[i][j] = dp[i - 1][j - 1] + 1;
8             else
9                 dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
10    cout << dp[n1][n2] << endl;
11 }
```

1.2 最长上升子序列

先说一种最普遍的方法，因为所求为子序列，所以这道题很容易想到一种线性动态规划。我们需要求最长上升子序列，为了上升我们肯定要知道我们当前阶段最后一个元素为多少，为了最长我们还要知

道当前我们的序列有多长。我们可以用前者来充当第一维描述：设 $F[i]$ 表示以 $A[i]$ 为结尾的最长上升子序列的长度，为了保证元素单调递增我们肯定只能从 i 前面的且末尾元素比 $A[i]$ 小的状态转移过来：

$$F[i] = \max_{0 \leq j < i, A[j] < A[i]} \{F[j] + 1\}$$

初始值为 $F[0] = 0$ ，而答案可以是任何阶段中只要长度最长的那一个，所以我们边转移边统计答案。

复杂度： $O(n^2)$

贪心加二分查找。

我们肯定要知道我们当前阶段最后一个元素为多少，还有当前我们的序列有多长。前两种方法都是用前者做状态，我们为什么不可以用后者做状态呢？

设 $F[i]$ 表示长度为 i 的最长上升子序列的末尾元素的最小值，我们发现这个数组的权值一定单调不降（仔细想一想，这就是我们贪心的来由）。于是我们按顺序枚举数组 A ，每一次对 $F[]$ 数组二分查找，找到小于 $A[i]$ 的最大的 $F[j]$ ，并用它将 $F[j + 1]$ 更新。

期望复杂度： $O(n \log n)$

```

1  int lengthOfLIS(vector<int>& nums) {
2      vector<int> tails; // tails[i]表示长度为i+1的LIS的最小末尾元素
3      for (int num : nums) {
4          // 使用二分查找找到第一个大于等于num的元素位置
5          auto it = lower_bound(tails.begin(), tails.end(), num);
6          if (it == tails.end()) {
7              // 如果num比所有元素都大，可以扩展LIS长度
8              tails.push_back(num);
9          } else {
10             // 否则替换第一个大于等于num的元素，保持tails数组的最小性
11             *it = num;
12         }
13     }
14     return tails.size(); // tails的大小就是LIS的长度
15 }

```

要同时求出最长公共子序列的长度和序列，首先初始化二维数组 $c[m + 1][n + 1]$ （存储长度）和 $b[m + 1][n + 1]$ （存储方向），将 c 的第一行和第一列初始化为 0。然后遍历字符串 X 和 Y ：当 $X[i - 1] = Y[j - 1]$ 时，令 $c[i][j] = c[i - 1][j - 1] + 1$ 并在 $b[i][j]$ 标记””；否则比较 $c[i - 1][j]$ 和 $c[i][j - 1]$ ，若 $c[i - 1][j] > c[i][j - 1]$ 则令 $c[i][j] = c[i - 1][j]$ 且标记”↑”，否则令 $c[i][j] = c[i][j - 1]$ 且标记”←”。最终 $c[m][n]$ 即为 LCS 长度。回溯时从 $b[m][n]$ 开始：若标记为””则递归 $b[i - 1][j - 1]$ 并记录 $X[i - 1]$ ；若为”↑”则递归 $b[i - 1][j]$ ；若为”←”则递归 $b[i][j - 1]$ 。最后将记录的字符反转即得 LCS 序列。

1.3 矩阵链乘法

给定 n 个矩阵的链 $\langle A_1, A_2, \dots, A_n \rangle$ ，其中矩阵 A_i 的维度为 $p_{i-1} \times p_i$

设一个由 n 个矩阵组成的序列的可选的括号化方案为 $P(n)$ 。则 $P(n)$ 递归式（卡特兰数）为：

Input: 字符串 $X = x_1x_2 \cdots x_m$, 字符串 $Y = y_1y_2 \cdots y_n$

Output: LCS 长度 $c[m][n]$, 方向矩阵 $b[1..m][1..n]$

```
1 for  $i \leftarrow 0$  to  $m$  do
2   |  $c[i][0] \leftarrow 0$ ;
3 end
4 for  $j \leftarrow 0$  to  $n$  do
5   |  $c[0][j] \leftarrow 0$ ;
6 end
7 for  $i \leftarrow 1$  to  $m$  do
8   for  $j \leftarrow 1$  to  $n$  do
9     if  $x_i == y_j$  then
10      |  $c[i][j] \leftarrow c[i-1][j-1] + 1$ ;
11      |  $b[i][j] \leftarrow "$ ↖ $"$ ;
12    else
13      if  $c[i-1][j] \geq c[i][j-1]$  then
14        |  $c[i][j] \leftarrow c[i-1][j]$ ;
15        |  $b[i][j] \leftarrow "$ ↑ $"$ ;
16      else
17        |  $c[i][j] \leftarrow c[i][j-1]$ ;
18        |  $b[i][j] \leftarrow "$ ← $"$ ;
19      end
20    end
21  end
22 end
23 return ( $c[m][n], b$ );
```

Algorithm 1: LCS-Length

$$P(n) = \begin{cases} 1 & n = 1 \\ \sum_{k=1}^{n-1} P(k)P(n-k) & n \geq 2 \end{cases}$$

其闭合形式解为:

$$P(n) = \frac{1}{n} \binom{2n-2}{n-1} = \frac{(2n-2)!}{n((n-1)!)^2} \approx \frac{4^n}{4\sqrt{\pi n^{1.5}}}$$

我们的求解过程分以下几步:

1. 描述最优解的结构特征。
2. 递归定义最优解的值。
3. 计算最优解的值。
4. 根据计算信息构造最优解。

Input: 方向矩阵 $b[1..m][1..n]$, 字符串 X , 位置 (i, j)

Output: LCS 序列

```
1 if  $i == 0$  or  $j == 0$  then
2   return;
3 end
4 if  $b[i][j] == "\searrow"$  then
5   PRINT-LCS( $b, X, i - 1, j - 1$ );
6   输出  $x_i$ ;
7 else if  $b[i][j] == "\uparrow"$  then
8   PRINT-LCS( $b, X, i - 1, j$ );
9 else
10  PRINT-LCS( $b, X, i, j - 1$ );
11 end
```

Algorithm 2: PRINT-LCS

定义 $A_{i:j}$ 为 $A_i A_{i+1} \cdots A_j$ 的乘积, 其中 $i \leq j$ 。如果 $A_{i:j}$ 可以继续分割 (即 $i < j$), 利用括号化分割成 $A_{i:k}$ 和 $A_{k+1:j}$, 其中 $i \leq k < j$ 。

$A_{i:j}$ 的括号化后的计算代价为:

$$\text{计算代价}(A_{i:j}) = \text{计算代价}(A_{i:k}) + \text{计算代价}(A_{k+1:j}) + \text{相乘代价}(A_{i:k}, A_{k+1:j})$$

关键性质:

- 子问题的最优括号化方案包含在原问题的最优括号化方案中
- 通过求出所有子问题的最优解, 就能组成原问题的最优解
- 在求解子问题的最优分割方案时, 需要对所有可能的分割点 k 进行考察

假设矩阵链乘问题不满足最优子结构性质, 即存在某个子问题的最优解不是构成原问题最优解的一部分。

考虑矩阵链 $A_1 A_2 \cdots A_n$ 的最优括号化方案, 设其在位置 k 处分割:

$$A_{1:n} = (A_{1:k})(A_{k+1:n})$$

根据假设, 存在以下两种情形之一:

1. $A_{1:k}$ 不是子问题 $A_{1:k}$ 的最优括号化方案
2. $A_{k+1:n}$ 不是子问题 $A_{k+1:n}$ 的最优括号化方案

以第一种情形为例 (第二种情形对称), 设存在更优的 $A_{1:k}^*$ 使得:

$$m^*[1, k] < m[1, k]$$

则可构造新的括号化方案:

$$A'_{1:n} = (A_{1:k}^*)(A_{k+1:n})$$

其总计算代价为:

$$\begin{aligned} m'[1, n] &= m^*[1, k] + m[k+1, n] + p_0 p_k p_n \\ &< m[1, k] + m[k+1, n] + p_0 p_k p_n \\ &= m[1, n] \end{aligned}$$

这与 $m[1, n]$ 是最优解的初始假设矛盾。因此, 原假设不成立, 矩阵链乘必须满足最优子结构性质。

定义 $m[i, j]$ 表示计算 $A_{i:j}$ 的乘积所需标量乘法的最少次数。计算 $A_1 A_2 \cdots A_n$ 的乘积所需标量乘法的最少次数可表示为 $m[1, n]$ 。

我们可以递归求解 $m[i, j]$:

- 当 $i = j$ 时, $A_{i:i} = A_i$, $m[i, i] = 0$, 其中 $i = 1, 2, \dots, n$
- 当 $i < j$ 时, 利用最优括号化方案:

$$m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j, \quad \text{其中 } i \leq k < j$$

分段函数形式表示为:

$$m[i, j] = \begin{cases} 0 & i = j \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\} & i < j \end{cases}$$

定义 $s[i, j]$ 表示 $A_{i:j}$ 的最优分割方案中的值 k , 即满足:

$$m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j$$

算法的时间复杂度是 $\Theta(n^3)$, 空间复杂度是 $\Theta(n^2)$

1.4 多源最短路径问题

1.5 0/1 背包问题

$$\begin{aligned} &\max \sum_{i=1}^n v_i x_i \\ &s.t. \begin{cases} \sum_{i=1}^n w_i x_i \leq C \\ x_i \in \{0, 1\} \quad i = 1, \dots, n \end{cases} \end{aligned}$$

设 $(y_1, y_2, \dots, y_n)$ 是上式的一个最优解, 下证 $(y_2, y_3, \dots, y_n)$ 是下面子问题的最优解:

$$\begin{aligned} &\max \sum_{i=2}^n v_i x_i \\ &s.t. \begin{cases} \sum_{i=2}^n w_i x_i \leq C - w_1 y_1 \\ x_i \in \{0, 1\} \quad i = 2, \dots, n \end{cases} \end{aligned}$$

Input: 矩阵维度序列 $p[0..n]$

Output: 最小乘法次数 $m[n, n]$ 和最优分割方案 $s[1..n-1, 2..n]$

```
1 for i ← 1 to n do
2   | m[i, i] ← 0 // 初始化对角线
3 end
4 for l ← 2 to n do
5   // l 为链长度
6   for i ← 1 to n - l + 1 do
7     j ← i + l - 1;
8     m[i, j] ← ∞;
9     for k ← i to j - 1 do
10      q ← m[i, k] + m[k + 1, j] + p[i - 1] × p[k] × p[j];
11      if q < m[i, j] then
12        m[i, j] ← q;
13        s[i, j] ← k // 记录最优分割点
14      end
15    end
16  end
17 end
```

Algorithm 3: MatrixChainOrder

考虑反证, 假设 $(z_2, z_3, \dots, z_n)$ 是上述子问题的最优解, 则:

$$\sum_{i=2}^n v_i z_i \geq \sum_{i=2}^n v_i y_i$$
$$w_1 y_1 + \sum_{i=2}^n w_i z_i \leq C$$

所以有

$$v_1 y_1 + \sum_{i=2}^n w_i z_i \geq \sum_{i=1}^n n v_i y_i$$

这说明 $(y_1, z_2, z_3, \dots, z_n)$ 是比 $(y_1, y_2, \dots, y_n)$ 更优的一个解, 矛盾!

背包问题的最优解能够在 $\Theta(nC)$ 的时间内和 $\Theta(C)$ 的空间内得到。

1.6 图像压缩问题

对于给定的 n 个像素点 $p_1, p_2, p_3, \dots, p_n$, 将其分割为连续的 m 个段 $S_1, S_2, S_3, \dots, S_m$ 。设第 i 个像素段 S_i 中, 有 $N[i]$ 个像素 ($1 \leq i \leq m$), 每个像素用 $b[i]$ 位表示 (比如 0-15 的数字, 要用 4 位表示)。

设 h_i 为分段 S_i 中最大灰度值所对应的二进制的位数, 即

$$h_i = \left\lceil \log \left(\max_{t[i]+1 \leq k \leq t[i]+N[i]} p_k + 1 \right) \right\rceil$$

Input: 分割矩阵 $s[1..n-1, 2..n]$, 矩阵编号 i, j

Output: 最优括号化方案

```
1 if  $i == j$  then
2   | 输出 " $A_i$ ";
3 end
4 else
5   | 输出 "(" ;
6   | PrintOptimalParens( $s, i, s[i, j]$ );
7   | PrintOptimalParens( $s, s[i, j] + 1, j$ );
8   | 输出 ")" ;
9 end
```

Algorithm 4: PrintOptimalParens

Input: $n \times n$ 维矩阵 $l[1..n, 1..n]$, 对于有向图 $G = (\{1, 2, \dots, n\}, E)$ 中的边 (i, j) 的长度是 $l[i, j]$

Output: 矩阵 D 使得 $D[i, j] = i$ 到 j 的距离

```
1  $D \leftarrow l$ ;
2 将输入矩阵  $l$  复制到  $D$  for  $k \leftarrow 1$  to  $n$  do
3   | for  $i \leftarrow 1$  to  $n$  do
4     | for  $j \leftarrow 1$  to  $n$  do
5       |  $D[i, j] = \min\{D[i, j], D[i, k] + D[k, j]\}$ ;
6     | end
7   | end
8 end
```

Algorithm 5: FLOYD 算法

由于 $p_k \leq 255$, 则 $h_i \leq b[i] \leq 8$, 需要用 3 位来表示 $b[i]$ 。我们给定 $N[i]$ 最多是 255, 这样需要 8 位来存储。像素段 S_i 需要的总内存空间是 $N[i] * b[i] + 11$ 。

像素序列 $\{p_1, p_2, p_3, \dots, p_n\}$ 所需存储空间是:

$$\sum_{i=1}^m N[i] \times b[i] + 11m$$

假设图像压缩问题不满足最优子结构性质, 即存在某个最优分段方案, 其中至少一个子段的分割不是局部最优的。

考虑像素序列 p_1 到 p_n 的最优分割方案 $S_1, \dots, S_m$, 设其中子段 S_k ($1 \leq k \leq m$) 不是该子区间的最优分割。则存在更优的分割 S_k^* 使得:

$$N[k] \times b[k] + 11 > N^*[k] \times b^*[k] + 11$$

Input: 物品集合 $U = \{u_1, u_2, \dots, u_n\}$, 体积 $s_1, s_2, \dots, s_n$, 价值 $v_1, v_2, \dots, v_n$, 背包容量 C

Output: $\sum_{u_i \in S} v_i$ 的最大总价值, 且满足 $\sum_{u_i \in S} s_i \leq C$, 其中 $S \subseteq U$

```

1 for  $i \leftarrow 0$  to  $n$  do
2   |  $V[i, 0] \leftarrow 0$ ;
3 end
4 for  $j \leftarrow 0$  to  $C$  do
5   |  $V[0, j] \leftarrow 0$ ;
6 end
7 for  $i \leftarrow 1$  to  $n$  do
8   for  $j \leftarrow 1$  to  $C$  do
9     // 如果是滚动数组的话, 这里需要是倒序的
10     $V[i, j] \leftarrow V[i - 1, j]$ ;
11    if  $s_i \leq j$  then
12      |  $V[i, j] \leftarrow \max\{V[i, j], V[i - 1, j - s_i] + v_i\}$ ;
13    end
14  end
15 end
16 return  $V[n, C]$ ;

```

Algorithm 6: KNAPSACK 算法

将原方案中的 S_k 替换为 S_k^* , 得到新方案的总存储空间:

$$\begin{aligned} \text{Total}^* &= \sum_{i \neq k} (N[i] \times b[i] + 11) + (N^*[k] \times b^*[k] + 11) \\ &< \sum_{i=1}^m (N[i] \times b[i] + 11) = \text{Total} \end{aligned}$$

这与原方案的最优性矛盾。因此, 最优分割方案中的每个子段必须自身是最优分割。

定义函数 $bm(i, j)$ 表示从像素 i 到 j 中像素灰度值 p_k 最大值所需的最少二进制位数:

$$bm(i, j) = \left\lceil \log \left(\max_{i \leq k \leq j} \{p_k\} + 1 \right) \right\rceil \quad (1 \leq i \leq n)$$

根据动态规划原理, 存储长度满足:

$$S[i] = \min_{1 \leq k \leq \min\{i, 256\}} \left\{ S[i - k] + k \times bm(i - k + 1, i) \right\} + 11 \quad (1)$$

```

1 //s[i]记录1-i像素所需的最小空间, I[i]记录1-i像素的最优划分的最后一个段的大小,
2 //b[i]由p[i]得出, 它记录p[i]需要几个比特来存储。
3 void compress2(int* p, int n, int* s, int* I, int* b)
4 {
5     int header = 11; //一个段需要3位来存储每个像素点需要几个比特来存储, 8位来存储这个像
                        素段有多少个像素点。

```



```

6      int Imax = 256; //N[i]的最大值
7      s[0] = 0;
8      //这个循环是为了计算s[i]数组，用I[i]数组保存在哪里分段的信息，其它数组、变量都是辅助。
9      for (int i = 1; i <= n; i++) {
10         //注意s[i]初始化为INT_MAX，而不是初始化为0，因为s[i]要寻找最小值
11         b[i] = length(p[i]);
12         int bmax = 0; //我们要算出在一个像素段中，最大的那个像素点需要用几个比特来存储，然后这个像素段中所有点都用那么多比特来存储。
13         s[i]=INT_MAX; //s[i]求的是最小值，所以初始化为大整数。
14         I[i];
15         for (int k = 1; k <= i && k <= Imax; k++) {
16             if (bmax < b[i - k + 1]) bmax = b[i - k + 1]; //如果像素段出现更大像素点，需要更大存储空间，刷新bmax
17             if (s[i] > s[i - k] + k * bmax) { //出现更小的s[i]则刷新
18                 s[i] = s[i - k] + k * bmax;
19                 I[i] = k; 记录分割位置：第1-i像素点的最后一个段有多少数字。
20             }
21         }
22         s[i] += header;
23     }
24 }

```

时间复杂度 $O(n)$

1.7 最大子段和

```

1  cin>>n;
2  for(int i=0;i<n;i++){
3      cin>>a;
4      f=max(f+a,a);
5      ans=max(f,ans);
6  }

```

1.8 多段图问题

- $P(i, j)$: 从 V_i 中结点 j 到汇点 t 的最短路径
- $COST(i, j)$: $P(i, j)$ 的路径成本
- $D(i, j)$: V_i 中结点 j 的最优决策（选择的后继节点）

$$\text{COST}(i, j) = \begin{cases} 0 & \text{当 } i = k \text{ (终止条件)} \\ \min_{\substack{l \in V_{i+1} \\ \langle j, l \rangle \in E}} \{c(j, l) + \text{COST}(i+1, l)\} & \text{当 } i < k \text{ (递推规则)} \end{cases}$$

- $BP(i, j)$: 从起点到阶段 V_i 节点 j 的最短路径
- $\text{BCOST}(i, j)$: 该路径的累计成本
- $D(i, j)$: 节点 j 选择的最优前驱节点

$$\text{BCOST}(i, j) = \begin{cases} 0 & \text{基础情况 (起点本身)} \\ \min_{\substack{l \in V_{i-1} \\ \text{边}(l, j) \in E}} [\text{BCOST}(i-1, l) + c(l, j)] & \text{递推情况} \end{cases}$$

- 将 n 个资源分配给 r 个项目
- 分配 j 个资源给项目 i 时, 获得净利 $N(i, j)$
- 目标: 找到使总净利最大的分配方案

1. 阶段划分:

- 构造 $r+1$ 段有向图 (r 个项目对应 r 个决策阶段 +1 个终止阶段)
- 段 i ($1 \leq i \leq r$) 表示项目 i 的分配决策

2. 节点设计:

- 源节点 $s = V(1, 0)$
- 中间节点 $V(i, j) = \begin{cases} \text{共分配 } j \text{ 个资源给项目 } 1, \dots, i-1, & 2 \leq i \leq r \\ j \in \{0, 1, \dots, n\} \end{cases}$
- 汇节点 $t = V(r+1, n)$

3. 边构造规则:

- 边形式: $\langle V(i, j), V(i+1, l) \rangle$, 满足 $j \leq l$ 且 $1 \leq i \leq r$
- 边权重:

$$w(\langle V(i, j), V(i+1, l) \rangle) = \begin{cases} N(i, l-j), & 1 \leq i < r \\ \max\{N(r, p) \mid 0 \leq p \leq n-j\}, & i = r \end{cases}$$

4. 求解方法:

- 最优解对应 s 到 t 的最大成本路径
- 通过将边权重取负 ($-N(i, k)$) 转换为多段图最短路径问题
- 动态规划递推式:

$$\text{COST}(i, j) = \max_{\substack{l \geq j \\ \langle V(i, j), V(i+1, l) \rangle \in E}} \{w(\langle V(i, j), V(i+1, l) \rangle) + \text{COST}(i+1, l)\}$$

Input: 成功概率数组 $p[1..n]$, 失败概率数组 $q[0..n]$, 关键字数量 n
Output: 期望代价矩阵 $e[1..n+1][0..n]$, 根节点矩阵 $root[1..n][1..n]$

```

1 for  $i \leftarrow 1$  to  $n+1$  do
2    $e[i][i-1] \leftarrow q[i-1]$ ;
3    $w[i][i-1] \leftarrow q[i-1]$ ;
4 end
5 for  $\ell \leftarrow 1$  to  $n$  do
6   for  $i \leftarrow 1$  to  $n-\ell+1$  do
7      $j \leftarrow i+\ell-1$ ;
8      $e[i][j] \leftarrow \infty$ ;
9      $w[i][j] \leftarrow w[i][j-1] + p[j] + q[j]$ ;
10    for  $r \leftarrow i$  to  $j$  do
11       $t \leftarrow e[i][r-1] + e[r+1][j] + w[i][j]$ ;
12      if  $t < e[i][j]$  then
13         $e[i][j] \leftarrow t$ ;
14         $root[i][j] \leftarrow r$ ;
15      end
16    end
17  end
18 end
19 return ( $e, root$ );

```

Algorithm 7: Optimal-BST

1.9 最优二分检索树问题

通过动态规划选择树结构，使得：

- 成功检索（命中关键字）和
- 不成功检索（未命中）的加权平均比较次数最少

$$\min \left(\sum_{i=1}^n p_i \cdot (\text{depth}(a_i) + 1) + \sum_{j=0}^n q_j \cdot \text{depth}(E_j) \right)$$

p_i : 关键字 a_i 的命中概率

q_j : 区间 E_j （失败检索区间）的未命中概率

$\text{depth}(\cdot)$: 结点在树中的深度

$$C(i, j) = \min_{i \leq k \leq j} \{C(i, k-1) + C(k+1, j) + W(i, j)\}$$

1.10 投资问题

给定 m 元资金和 n 个投资项目，每个项目的效益函数为 $f_i(x)$ ，表示投入 x 元到第 i 个项目的收益。目标是通过资金分配最大化总效益：

$$\max \sum_{i=1}^n f_i(x_i) \quad \text{s.t.} \quad \sum_{i=1}^n x_i = m, \quad x_i \in \mathbb{N}$$

定义状态函数：

$F_k(x)$ = 将 x 元分配给前 k 个项目的最大效益

递推关系与边界条件：

$$F_k(x) = \max_{0 \leq x_k \leq x} \{f_k(x_k) + F_{k-1}(x - x_k)\}, \quad k > 1$$

$$F_1(x) = f_1(x)$$

1.11 本章作业

1.11.1 改进 floyd 算法

请修改求解所有点对的 Floyd 算法，使它可以检测到负权圈（一个负权圈是它的总长度为负的圈）。

- **负权圈问题：**当图中存在总权重为负的环路时，Floyd 算法会陷入无限更新状态。例如，若路径 $i \rightarrow j$ 包含负权圈，每次绕行该圈都会使总路径权重减小，导致算法无法收敛。

数学表现为：

$$\exists \text{环 } C : \sum_{e \in C} w(e) < 0 \Rightarrow \lim_{k \rightarrow \infty} dist^{(k)}[i][j] = -\infty$$

- **最优子结构破坏：**动态规划要求子问题的最优解能构成全局最优解。负权圈使得通过无限绕行可不断降低路径权重，导致：

局部最优 $\nrightarrow$ 全局最优

$$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j]) \quad (\text{不收敛})$$

改进思路：在算法完成后检查距离矩阵对角线元素，满足：

$$\exists i \in V : dist[i][i] < 0 \Rightarrow \text{存在包含顶点 } i \text{ 的负权圈}$$

1.11.2 有向图的所有点对最短路径问题

设 G 是一个有向图，其中每一条边有一个非负的长度，如果两点之间无边，令其长度为无穷。求图中每个顶点到其他顶点的最短路径。请证明该问题满足最优子结构性质。

假设最短路径问题不满足最优子结构性质，则存在以下情形：

1. **假设：**设 $p = \langle v_1, v_2, \dots, v_k \rangle$ 是从 v_1 到 v_k 的最短路径，但存在子路径 $p_{ij} = \langle v_i, v_{i+1}, \dots, v_j \rangle$ 不是 v_i 到 v_j 的最短路径。

2. **构造矛盾：**

- 存在更优路径 p'_{ij} 满足 $w(p'_{ij}) < w(p_{ij})$
- 构造新路径 $p' = \langle v_1, \dots, v_{i-1} \rangle \circ p'_{ij} \circ \langle v_{j+1}, \dots, v_k \rangle$
- 计算总权重：

$$w(p') = w(p) - w(p_{ij}) + w(p'_{ij}) < w(p)$$

3. **矛盾结论：**这与 p 是最短路径的初始假设矛盾，故原命题得证。

1.11.3 k-shattering 问题

问题定义 给定边带权值的完全二叉树 $T = (V, E, w)$, 其中 $w : E \rightarrow \mathbb{R}^+$ 。一个 k -shattering 集合是边集 $S \subseteq E$ 满足:

- 删除 S 后, T 被划分为 m 棵子树 $\{T_1, \dots, T_m\}$
- 每棵子树 T_i 的节点数 $|V(T_i)| \leq k$

求使 $\sum_{e \in S} w(e)$ 最小的 k -shattering 集合。

```

1 Function kShatteringDP( $T, w, k$ )
2    $n \leftarrow |V(T)|$ ;
3    $\text{dp}[1..n][1..k] \leftarrow \infty$ ;
4   for  $i \leftarrow n$  downto  $\lfloor n/2 \rfloor + 1$  do
5     for  $m \leftarrow 1$  to  $k$  do
6        $\text{dp}[i][m] \leftarrow 0$ ; // 叶子节点初始化
7     end
8   end
9   for  $i \leftarrow \lfloor n/2 \rfloor$  downto 1 do
10    for  $j \leftarrow 1$  to  $k$  do
11      if  $j = 1$  then
12         $\text{min\_left} \leftarrow \min_{1 \leq a \leq k} \text{dp}[2i][a]$ ;
13         $\text{min\_right} \leftarrow \min_{1 \leq b \leq k} \text{dp}[2i+1][b]$ ;
14         $\text{dp}[i][1] \leftarrow w(i, 2i) + w(i, 2i+1) + \text{min\_left} + \text{min\_right}$ ;
15      else
16         $\text{option1} \leftarrow w(i, 2i) + \min_{1 \leq a \leq k} \text{dp}[2i][a] + \text{dp}[2i+1][j-1]$ ;
17         $\text{option2} \leftarrow w(i, 2i+1) + \min_{1 \leq a \leq k} \text{dp}[2i+1][a] + \text{dp}[2i][j-1]$ ;
18         $\text{option3} \leftarrow \min_{1 \leq a \leq j-2} (\text{dp}[2i][a] + \text{dp}[2i+1][j-a-1])$ ;
19         $\text{dp}[i][j] \leftarrow \min(\text{option1}, \text{option2}, \text{option3})$ ;
20      end
21    end
22  end
23  return  $\min_{1 \leq j \leq k} \text{dp}[1][j]$ ;

```

Algorithm 9: 最小代价 k -shattering 算法

- 状态定义: $\text{dp}[i][m]$ 表示以 i 为根的子树保留 m 个节点的最小代价
- 边界条件:

$$\text{dp}[i][m] = 0 \quad \forall i > \lfloor n/2 \rfloor$$

- 状态转移:

$$\text{dp}[i][j] = \begin{cases} w_{i,2i} + w_{i,2i+1} + \min_a \text{dp}[2i][a] + \min_b \text{dp}[2i+1][b] (\text{只有根}) & j = 1 \\ \min \begin{cases} w_{i,2i} + \min \text{dp}[2i][a] + \text{dp}[2i+1][j-1] (\text{断左}) \\ w_{i,2i+1} + \min \text{dp}[2i+1][a] + \text{dp}[2i][j-1] (\text{断右}) \\ \min_a (\text{dp}[2i][a] + \text{dp}[2i+1][j-a-1]) (\text{全留}) \end{cases} & j > 1 \end{cases}$$

- 时间复杂度: $O(nk^2)$

最小代价: $\min(16, 9) = 9$, 对应删除边集 $\{\langle b, d \rangle, \langle b, e \rangle, \langle a, c \rangle\}$ 。

Input: 效益函数 $f[1..n][0..m]$, 总资金 m , 项目数 n
Output: 最大效益值 $F[n][m]$, 最优分配方案 $alloc[1..n]$

```

1 // 初始化阶段：处理单个项目情况
2 for  $x \leftarrow 0$  to  $m$  do
3      $F[1][x] \leftarrow f[1][x]$ ;
4      $tempAlloc[1][x] \leftarrow x$ ;
5     // 记录初始分配方案
6 end
7 // 动态规划递推阶段
8 for  $k \leftarrow 2$  to  $n$  do
9     for  $x \leftarrow 0$  to  $m$  do
10          $F[k][x] \leftarrow 0$ ;
11         for  $x_k \leftarrow 0$  to  $x$  do
12              $current \leftarrow f[k][x_k] + F[k-1][x-x_k]$ ;
13             if  $current > F[k][x]$  then
14                  $F[k][x] \leftarrow current$ ;
15                  $tempAlloc[k][x] \leftarrow x_k$ ;
16                 // 保存当前最优分配
17             end
18         end
19     end
20 end
21 // 回溯最优分配方案
22  $remaining \leftarrow m$ ;
23 for  $k \leftarrow n$  downto 1 do
24      $alloc[k] \leftarrow tempAlloc[k][remaining]$ ;
25      $remaining \leftarrow remaining - alloc[k]$ ;
26 end
27 return  $(F[n][m], alloc)$ ;

```

Algorithm 8: Investment-Allocation-DP